VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

BASIC PROGRAMMING

Lecture 3: Program Control Flows – Making Decisions

Dr. NGUYEN Hoang Ha
Dr. NGUYEN Minh Huong
Dr. LE Huu Ton

Lecture 3:

- Structured Programming
- Sequential Structure
- Brach Structure
- Loop Structure
- Practical time

Structured Programming

A statement in C

- The simplest statement:
  - An expression followed by a semicolon.
  - Example:

```
x = 2;              /* an assignment statement */
x = 2+3;            /* another assignment statement */
2+3;                /* has no effect---will be discarded by smart compilers */
puts("hi");         /* a statement containing a function call */
root2 = sqrt(2);    /* an assignment statement with a function call */
```

- We can improve the organization of code by more complicated structures, e.g. : Jumping statements:
  - return
  - break
  - continue
  - goto

The “goto” example

```c
#include <stdio.h>
int main()
{
    int i = 1;
    loop:
    printf("%d \n",i);
    i++;
    if(i<=10)
        goto loop;
    return 0;
}
```

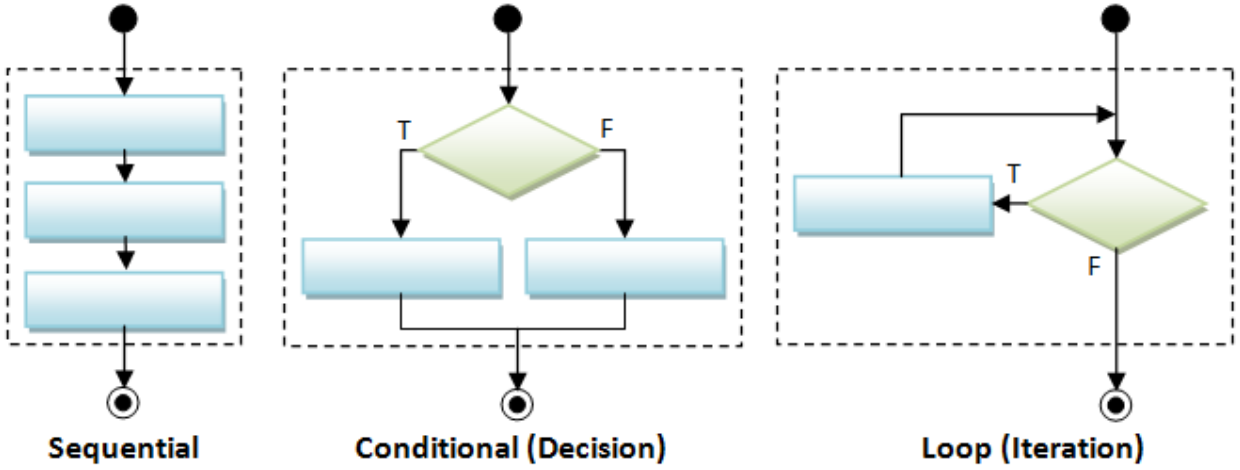
A statement in C (cont')

- Compound statements:
  - Multiple statements are grouped into a block by braces {...}
  - Syntactically equivalent to a single statement
  - No semicolon after the right brace
  - Example:

```
if ( i > 0 )
{
    line[i] = x;
    x++;
    i--;
}
```

Structured Programming

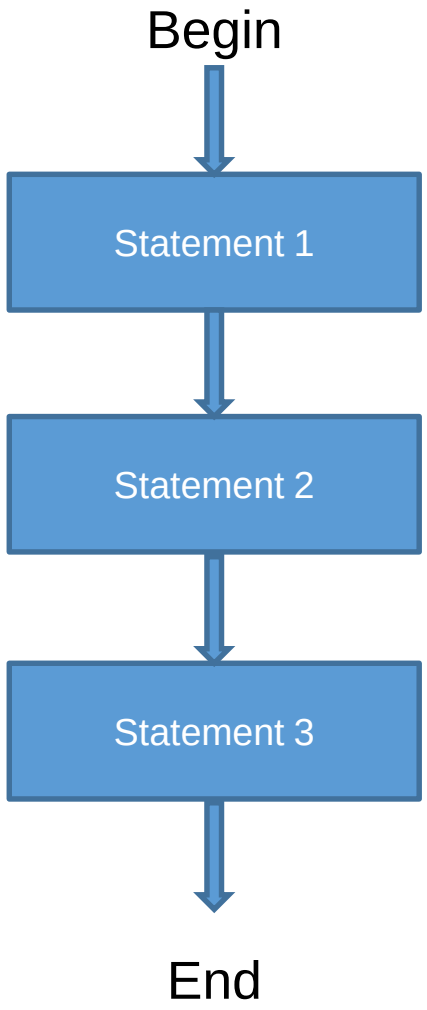
- A program consists of instructions by instructions compiled and executed by CPU.
- In structed programming, instructions is executed in a serial and structured manner.
- Structures:
  - Sequence
  - Conditional branches
  - Loop



Sequential Structure

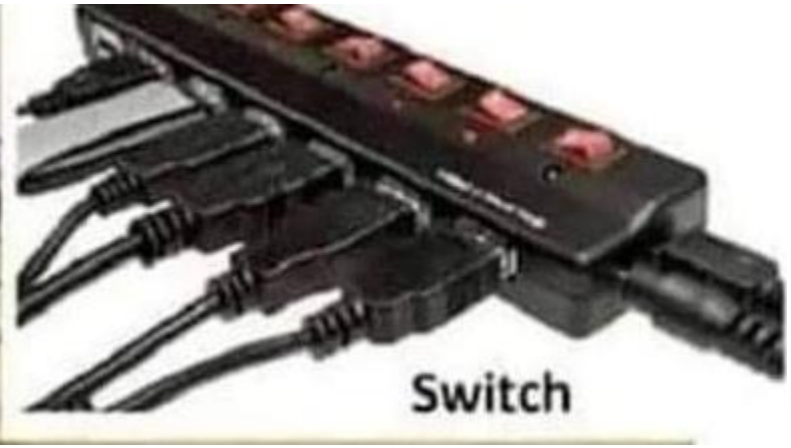
Sequential Structure

- Statements are executed in the same order specified in program
- All statements are executed exactly once



Structures be like...









Branch structure

Decision making

- In daily talking:
  - If I feel well, (then) I will attend Basic Programming lecture. Otherwise, I will stay at home.

- Decision making in a program:
  - Choosing to execute one set of statements rather than others
  - Involves conditional expression or logical expression

Conditional/Logical Expression

- An expression is either true (1) or false (0)
- Simple conditional expression:
  - An expression using relational operators:
  - Example:

```
int a = 4;
int b = 5;
if (a<b)
{
   printf("You have no homeworks");
}
```

Conditional/Logical Expression

- Combination of simple conditional expressions using logical operators
  - Example:

```
int a = 4;
int b = 5;
if ((a<5) && (b>10))
{
   printf("You have no homeworks");
}
```

Conditional/Logical Expression

- Relational operators:
  - Producing a Boolean result (true or false)
  - Boolean data type:
    - _Bool

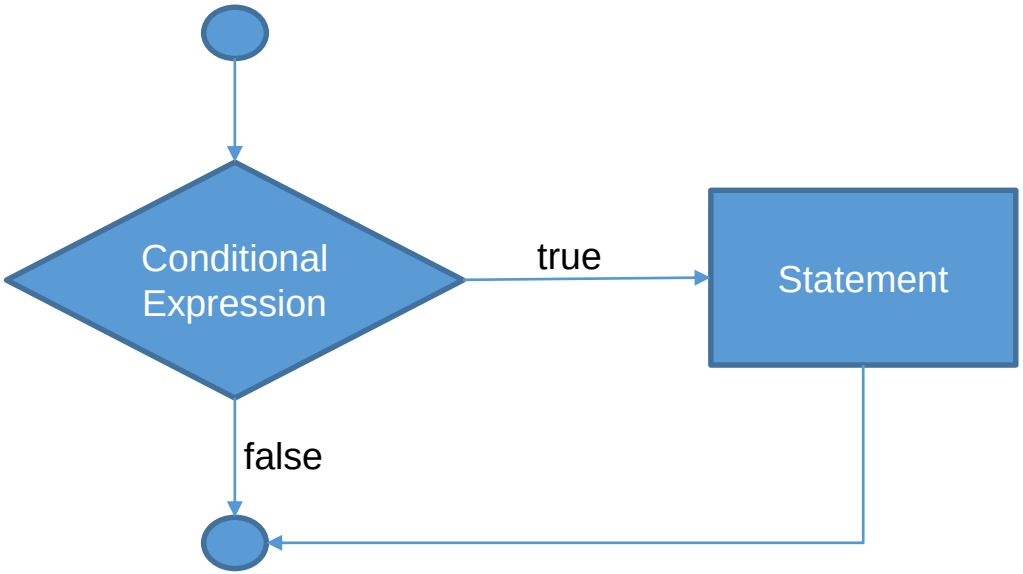
```
_Bool result = 4<5;
```

    - If the header file <stdbool.h> is included: bool

```
bool result = 4<5;
```

Basic if Statement

- A statement to make decision



Basic if Statement (cont')

Syntax:

```
if (expression)
        Statement1;
Next_statement;
```

Or

```
if (expression) Statement;
```

Basic if Statement (cont')

- Example:

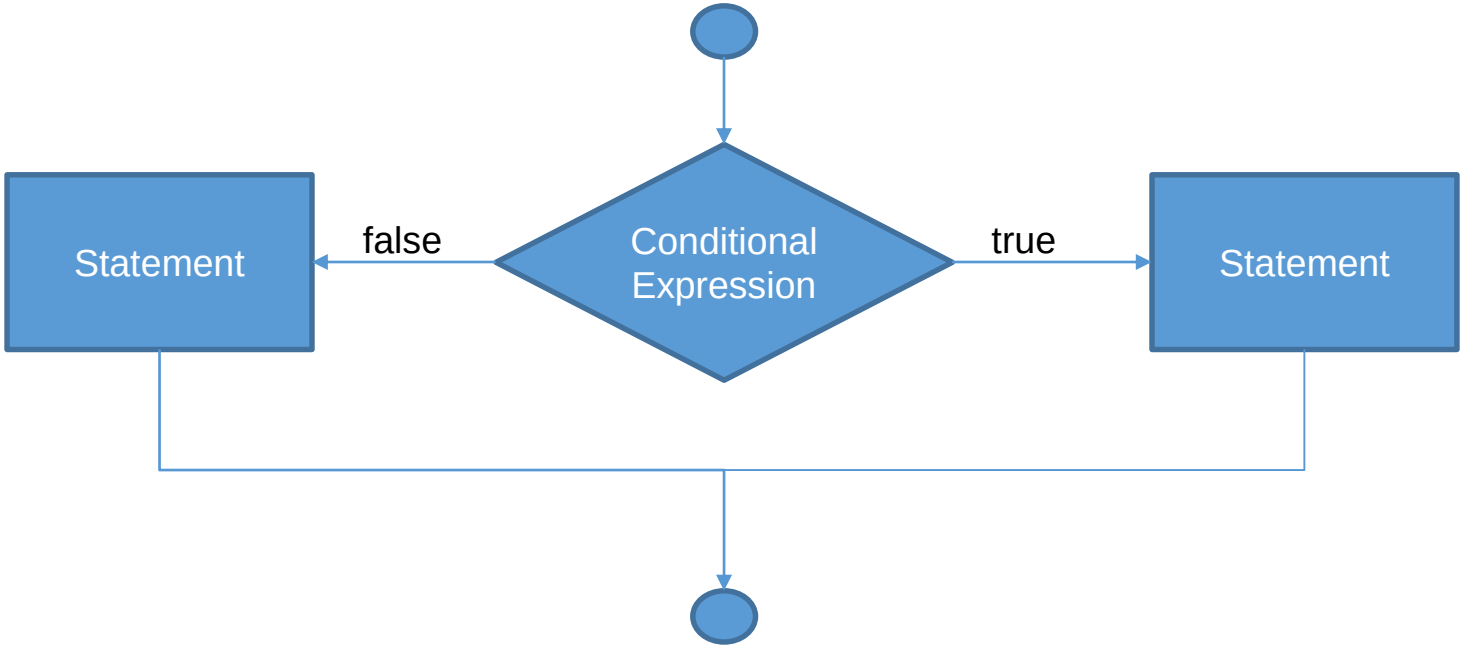
```c
#include <stdio.h>
int main(void)
{
   int number = 0;
   printf("Enter an integer between 1 to 10: \n");
   scanf("%d",&number);
   if (number > 10)
   {
      printf("The number %d is greater than 10 \n", number);
   }
   if (number <= 10)
   {
      printf("The number is %d \n", number);
   }
   return 0;
}
```

Exercises

1. Write a program to get the absolute value of a number
2. Write a program to sort out two numbers from low to high values

If-Else Statement

- Extend the if statement



If-Else Statement

- Syntax

```
if (expression)
Statement 1;
Else
Statement 2;

Next_statement;
```

If-Else Statement

- Example:

```c
#include <stdio.h>
int main(void)
{
   int number = 0;
   printf("Enter an integer between 1 to 10: \n");
   scanf("%d",&number);
   if (number > 10)
   {
      printf("The number %d is greater than 10 \n", number);
   }
   else
   {
      printf("The number is %d \n", number);
   }
   printf("The End \n");
   return 0;
}
```

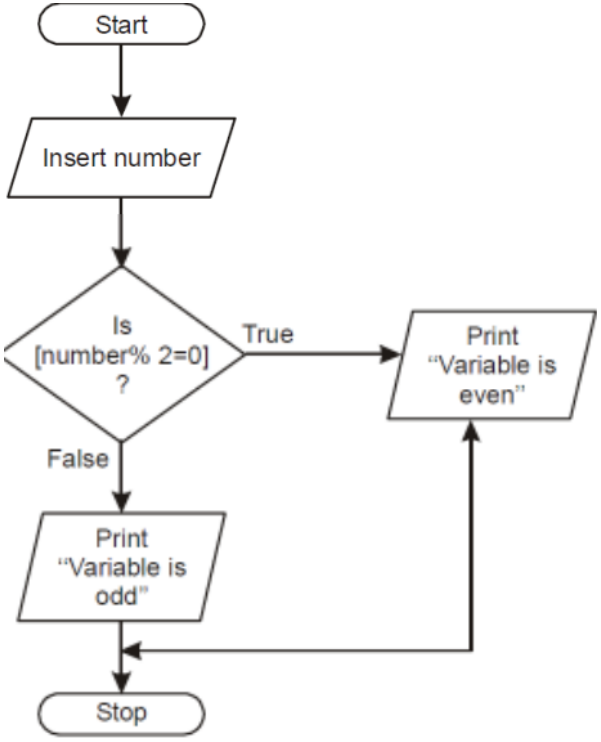
Exercises

Write a program to check if a given number is even or odd

```c
#include <stdio.h>
int main() {
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);

    // true if num is perfectly divisible by 2
    if(num % 2 == 0)
        printf("%d is even.", num);
    else
        printf("%d is odd.", num);

    return 0;
}
```



Nested if statements

- Ifs within ifs are called nested ifs
- Nested ifs are used to test more than one condition

Nested if statements

- Syntax:

```
if (expression1)
{
   StatementA;
   if (expression 2)
   {
      StatementB;
   }
   else
   {
      StatementC;
   }
}
else
{
   StatementD;
}
```

Nested if statements

- Example:
  - Check a given number to be odd or even. If it's even, is half of it is even?

Nested if statements

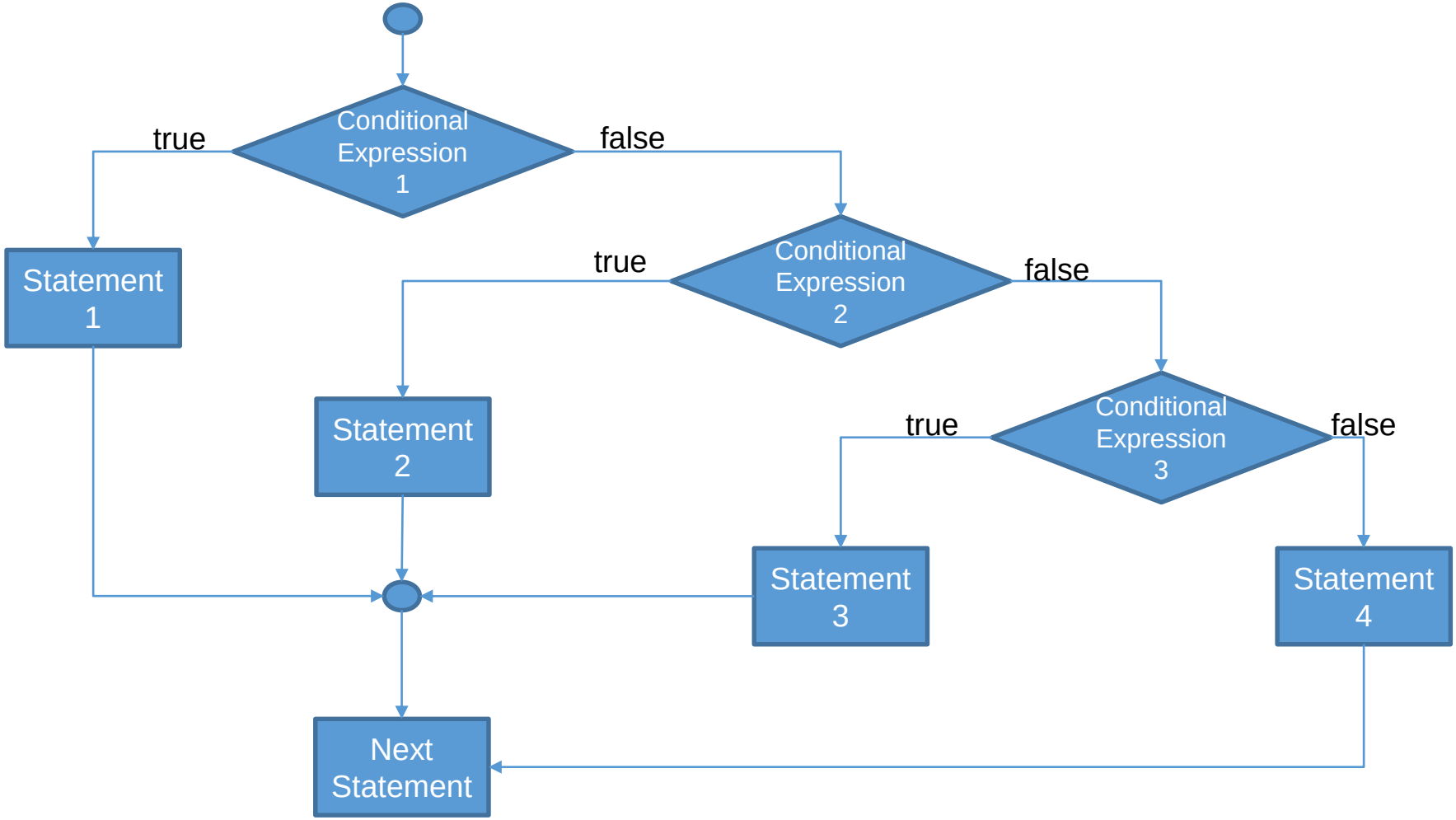
```c
#include <stdio.h>
int main(void)
{
    int number = 0;
    printf("Enter an integer between 1 to 10: \n");
    scanf("%d",&number);

    if (number%2 == 0)
    {
        printf("The number %d is even \n", number);
        if ((number/2)%2==0)
        {
            printf("Half of %d is even \n", number);
        }
    }
    else
    {
        printf("The number %d is odd \n", number);
    }
    return 0;
}
```

If - else if - else statements

- Multiple choice questions
- Two ways to make choices:
  - Using if - else if -…- else statements
  - Using switch statement

If - else if - else statements



If - else if - else statements

Syntax:

```
if (expression1)
{
    Statement1;
}
else if (expression2)
{
    Statement2;
}
else if (expression3)
{
    Statement3;
}
else
{
    Statement4;
}

Next statement;
```

Switch statement

- Enables you to choose one course of action from a set of possible actions
- Syntax:

```
switch(integer_expression)
{
    case constant_expression_1:
        statement_1;
        break;
        …
    case constant_expression_n:
        statement_n;
        break;
    default:
        statements;
}
```

Loop Structure

Definition of Loops

- A mechanism to repeat blocks of statements multiple times until some conditions are met
- Two main kinds of loops:
  - Counting loops
  - Conditional loops

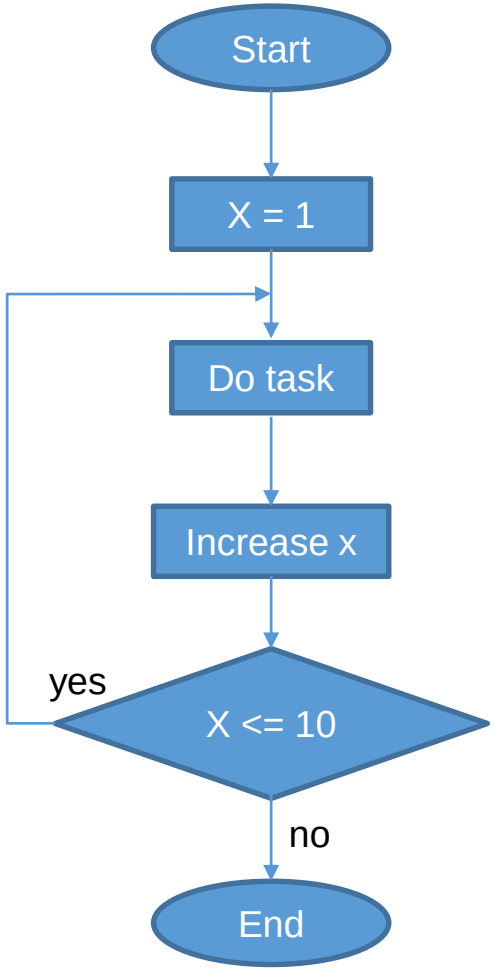
Counting Loop – For loop

- Execute a block of statements a given number of times
- Syntax:

```
for (init; condition; increment)
{
    statements;
}
```

Counting Loop – For loop

- *Init*: Initial iteration variable
- *Expression*: logical expression to check the iteration condition
- *Increment*: change the value of the iteration variable



Counting Loop – For loop

- Example:

```
#include <stdio.h>
int main(void)
{
    for (int x =1; x <=10; ++x)
        printf("\n%d", x);
    return 0;
}
```

Multiple iterations

- Example:

```
for(int i = 1, j = 2 ; i<=5 ; i++, j = j+2)
printf("\n %5d", i*j);
```

Multiple conditions

- Example:

```
for(int i = 0,j=0; (i<10)&&(j<20); i++,j+=5)
printf("\n %d", i);
```

Increment and Decrement Operators

- Increment operator:
  - Adds 1 to the variable it acts on: ++
  - Prefix ++i: increasing the variable before its value is used
  - Postfix i++: increasing the variable after its value is used
  - Example:

```
int count = 5;
total = ++count + 6;//total=12

int count = 5;
total = count++ +6;//total=11
```

Increment and Decrement Operators

- Decrement operator:
  - Subtract 1 to the variable it acts on: ++
  - Prefix: --i
  - Postfix: i--
  - Example:

```
int count = 5;
total = --count + 6;//total=10

int count = 5;
total = 6+ count--;//total=11
```

Break statement

- Stop executing the code within the current loop and continue the first statement following the loop
- Example:

```
#include <stdio.h>

int main()
{
	int x=5;
	for (int i =1; i<=x; i++)
	{
		if (i==3)
			break;
		printf("%d \n",i);
	}
	return 0;
}
```

Nested for loop

- The inner loop completes all its iterations for each iteration of the outer loop
- Example:

```
#include <stdio.h>
int main()
{
     int x;
     printf("Enter a number to sum ");
     scanf("%d",&x);
     int sum;
     for (int i =1; i<=x; i++)
     {
          sum =0;
          for (int j =1 ; j<=i; j++)
          {
                    sum = sum +j;
          }
          printf("%d \t %d \n",i,sum);
     }
     return 0;
}
```

Conditional Loop

- While loop
- do…while loop

While loop

- Repeating a set of statements until a condition is met
- Check the condition before doing statements
- Syntax:

```
while (expression)
{
    statements;
}
Next statements;
```

Do...while loop

- The condition is tested at the end of the loop
- Syntax:

```
do
{
    statements
}
while (expression);
```

Do...while loop

- Example:

```
#include <stdio.h>
int main ()
{
   int number = 0; /* The number to be reversed */ int rebmun = 0; /* The reversed n
umber */
   int temp = 0; /* Working storage */
   /* Get the value to be reversed */
   printf("\nEnter a positive integer: ");
   scanf(" %d", &number);
   temp = number;
   do
   {
      rebmun = 10*rebmun + temp % 10; /* Add the rightmost digit */
      temp = temp/10; /* Remove the rightmost digit */
   }while(temp); /* Continue while temp>0 */

   printf("\nThe number %d reversed is %d rebmun ehT\n", number, rebmun );
   return 0;
}
```

The continue statement

- Use case:
  - When you want to skip a current iteration and continue the next one
- Example:

```
for(int day = 1; day<=7 ; ++day)
{
    if(day == 4)
    continue;
    /* Do something useful with day */
}
```

The goto statement

- To escape nested loops completely
- Example:

```
for(int day = 1; day<=7 ; ++day)
{
   for (int time =1; time <=24; ++time)
   {
      if(day == 3 && time == 10)
      goto out;
      /* Do something useful with day */
      printf("day = %d time = %d \n", day, time);
   }
}
out: printf("Let's go out");
```

Practical time

- Design a program for Simple-Simon game:
  - Memory test game
  - Display a sequence of digit on the screen in short time
  - Memorize them, when they disappears, you must enter exactly identical sequence of digit
  - Each time you succeed, repeat the process with longer sequence